

Justus Liebig University Giessen

Faculty 06

Department of Sports Science



**Reliability of Horizontal and Vertical motion capture
using integrated smartphone sensor fusion**

Module : Measurement Projects in Movement Analytics

Supervisor : Mathias Reiser

Author : Philipp Ewen

Khouloud Chtioui

Eliakim kipruto Barmao

Submission date : 27/02/2026

Table of Contents

1. Introduction	2
2. The Sensor fusion algorithm	2
2.1 Data Import, Preprocessing and Synchronisation	2
2.2 Bias Elimination and Filtering	3
2.3 Zero Velocity Update and Madgwick Attitude and Heading Reference System	3
2.4 Extended Kallmann Filter	5
2.5. Elevation calculation	6
2.6 Weighted Distance calculation	8
2.7 KPI Calculation and saving	8
3. Methods	9
3.1 Measurement Setup	9
3.2 Ground Truth Establishment	10
3.3 Measurement Phases and Statistical Methodology	11
4. Results	12
4.1 Quantitative Evaluation Against Ground Truth	12
4.2 Bland–Altman Agreement Analysis	14
5. Discussion	16
6. Limitation	19
7. Conclusion	19
8. Literature	22

1. Introduction

Accurate estimation of distance and elevation is essential for performance analysis in middle-distance running. Smartphones provide positioning data through Global Navigation Satellite Systems (GNSS), such as GPS, but their accuracy is highly dependent on environmental conditions. Position errors typically range from 3–5 m under open sky and can increase to 10–50 m in obstructed environments such as urban areas (Hu et al., 2023). This variability makes GNSS-only approaches insufficient for reliable distance and elevation estimation.

To improve measurement accuracy, modern systems increasingly rely on sensor fusion techniques that combine GNSS with additional onboard sensors, including inertial measurement units (IMU), gyroscopes, magnetometers, and barometers (Groves, 2013). While each sensor has inherent limitations, integrating multiple data sources allows complementary strengths to be utilized while reducing individual weaknesses (Grewal et al., 2007).

Therefore, this project integrated data from multiple sensors GNSS (positioning), Inertial Measurement Unit (IMU) (force), Magnetometer (heading), Gyrometer (yaw, pitch, and rotation) and Barometer (pressure-based elevation). To tackle this task a data processing pipeline, based on scientifically proven methods was created, to try and replicate a sensor fusion pipeline that might look like what is used in Google, strava or garmin software. Following meticulous calibration, the algorithm was tested, analyzed, and ultimately evaluated to be more precise than estimations based on a single sensor alone.

2. The Sensor fusion algorithm

2.1 Data Import, Preprocessing and Synchronisation

The algorithm consists of eight major steps, starting with data acquisition, synchronization, and NaN elimination. Since the various sensors operate at different frame rates, it is crucial to align all data on a common timeline. Because NaNs (Not-a-Number values) can easily corrupt calculations and estimations, the data is prepared using linear interpolation, synchronization, and "fillmissing" functions. Instead of relying on standard GPS elevation data, a DGM1 mosaic (Digital Terrain Model with 1-meter resolution) was utilized to provide superior reliability. This data was obtained through official state portals, such as those for Hesse or Bavaria, and served to replace the more error-prone GPS altitude readings with high-precision terrain data to get a reliable elevation estimation for the current position. To extract the DGM Elevation for the current GNSS position, a custom Python script was developed with the assistance of ChatGPT. This approach was chosen because the corresponding MATLAB toolbox for this specific function is quite expensive, and a cost-free alternative was preferred. In the script, the longitude and

latitude coordinates were extracted from the GPS signal, and the DGM mosaic was loaded. The GPS coordinates were then projected onto the mosaic, allowing the exact elevation data to be extracted and saved into a CSV file titled "GPS_DGM.csv".

2.2 Bias Elimination and Filtering

Next up, the data is preprocessed by eliminating sensor bias and filtering noise. This is achieved by identifying a stationary timeframe. The thresholds for detecting these stationary phases are set to approximately 9.81 m/s^2 for the accelerometer (reflecting gravity) and $1^\circ/\text{s}$ for the gyroscope. The algorithm searches for a window of at least one second in which at least half of the samples fulfill these criteria. To avoid artifacts caused by initial handling or sensor stabilization, this search only begins after the first 10 seconds of the recording. If the dataset lacks a naturally occurring stationary window, the algorithm employs a fallback mechanism: it identifies a segment of 100 samples with the lowest variance. The mean values from this window are then deducted from the entire dataset to correct the constant bias of the IMU and gyroscope. Afterward, a Butterworth low-pass filter with a 20 Hz cutoff frequency is applied to the IMU data to reduce high-frequency "impact noise" caused by foot strikes during running. Finally, the magnetometer data is normalized to a unit vector. For subsequent orientation estimations, the absolute magnitude of the magnetic field is irrelevant; only the precise direction of the vector is required to determine the heading.

2.3 Zero Velocity Update and Madgwick Attitude and Heading Reference System

The Zero Velocity Update (ZUPT) and the Madgwick Attitude and Heading Reference System (AHRS) are implemented based on the foundational scientific work of Sebastian Madgwick (2010). To implement this step, an open-source MATLAB class provided by x-io Technologies was used. The complete integration of this object-oriented structure into the processing pipeline was made with the help of Gemini to ensure that the complex mathematical functions for sensor fusion were implemented correctly. The LLM was prompted with the specific goal of the algorithm and the question of how to best integrate the class-based MATLAB function into the existing code architecture.

The primary goal here is to merge the data from the accelerometer, gyroscope, and magnetometer into a single, unified reference system that provides the best estimate of the device's orientation. The algorithm begins by initializing a list of potential values for the system parameters (beta). These parameters balance the influence of the gyroscope against the accelerometer and magnetometer. Simultaneously, specific thresholds are set for the gyroscope and accelerometer to detect stationary phases. Identifying these points is vital to resetting calculation errors that would otherwise accumulate. Without the stabilization

from the Madgwick filter and the periodic resets from ZUPT, the system would suffer from two primary types of mathematical degradation: Orientation Drift and Velocity Integration Drift. To find the orientation, the algorithm integrates the angular velocity from the gyroscope over time. However, every sensor has a small, constant offset or bias, even when perfectly still, it might report a tiny rotation (e.g., $0.01^\circ/\text{s}$). While this seems negligible, the integration process adds this error up every millisecond. The same of course also applies to the velocity based on IMU sensors.

The core of this process is the MadgwickAHRS class, which processes the raw sensor data in two steps. First, it calculates a so-called quaternion derivative from the gyroscope data. In mathematics, a derivative represents a rate of change, in our case, it is the algorithm's first guess of how the device's orientation is changing from one millisecond to the next. This derivative represents how fast and in which direction the gyrometer is turning, but it has no way of knowing if its overall path is still aligned with the real world. To prevent this guess from drifting over time, the algorithm uses a Gradient Descent step as a corrective measure. Mathematically, Gradient Descent is an optimization tool used to find the minimum of an error function. In our project, we are searching for the optimal orientation where the difference between our gyroscope's guess and the physical reality (gravity and magnetic North) is as small as possible. The "Gradient" is similar to a slope of a hill. By calculating this slope, the algorithm knows exactly which direction the Error becomes less. By constantly moving toward this minimum, the Gradient Descent step directs our floating gyroscope guess back to the stable anchors of Earth's gravity and magnetic field.

The final result of this fusion is a normalized quaternion. While degrees (Euler angles) are more easily understandable, the algorithm uses quaternions because they represent rotation in 4D space. This is essential to avoid Gimbal Lock. In a 3D system using standard degrees, a rapid flip of the smartphone could cause two axes to overlap, making the system unable to distinguish between a side-to-side tilt and a rotation. Quaternions treat the rotation as a single, fluid movement around an axis, ensuring the math never freezes even during the most intense movements of a run. Once this stable 4D orientation is established, it is converted back into readable Euler angles (Roll, Pitch, and Yaw), providing a clean and reliable foundation for the subsequent tracking phases.

Simultaneously, the Zero Velocity Update (ZUPT) test is performed. The script monitors the shakiness of the sensors. If the noise from the accelerometer and gyroscope stays below our chosen thresholds, the algorithm marks that moment as Zero Velocity. To ensure these detections are reliable, a moving median filter is applied. This acts like a stability checker that requires a stationary state to last for a certain number of samples before it is officially counted, preventing random noise from being mistaken for a foot strike. To find the most accurate configuration, the script performs a systematic Grid Search, simulating the entire movement for every possible combination of filter strengths and stillness thresholds. Each result is evaluated by a custom Scoring Function that acts as a quality judge. A parameter set

receives a Perfect Score by proving three things: the tilt angles (Roll and Pitch) don't jitter, the compass heading (Yaw) remains stable, and the number of ground contacts matches human physiology. Specifically, we aim for a ZUPT ratio of approximately 12%. This target is based on the biomechanics of running: unlike walking, where the foot is on the ground for a long time, a runner's stance phase is very brief. Research in inertial navigation (Skog et al., 2010) suggests that a strict threshold capturing only the most stable 10-15% of the gait cycle is optimal to prevent artifacts from being mistaken for zero-velocity moments.

By tracking and keeping only the settings with the absolute best score, the algorithm ensures a stable and robust foundation. Finally, the script applies a 20% safety margin to the thresholds, making them 20% stricter than the "best" values found during the search. This margin is a critical protection against false positives. During a high-intensity run, there are many "near-still" moments (for a sensor), such as the brief pause at the peak of a leg swing, that could trick a loose filter into triggering a Zero Velocity Update. By tightening the requirements by 20%, we ensure that a velocity reset only happens when the foot is locked on the ground. This prevents the subsequent Extended Kalman Filter (EKF) from receiving false data, which would otherwise lead to massive jumps in the distance estimation.

2.4 Extended Kallmann Filter

In the fourth step, we implement the Extended Kalman Filter (EKF). This is the most critical part of the entire pipeline, as it merges the data from the ZUPT, the AHRS, and the GPS into a single coordinate system as described in grooves chapter 14.2 and 15.3.2 (2013, pp. 583 - 585, pp. 624 - 626). The implementation of all the steps for the extended kallmann filter was developed using coding and logic support from Gemini, which was specifically prompted using the methodology and mathematical derivation sections of the works of grooves (2013) to ensure the transition matrices and update logic were correctly translated into MATLAB code.. The GPS data must first be converted to calculate the exact distance of the run in meters. While standard GPS uses a latitude/longitude system (measuring the Earth in degrees), our calculations require a flat, metric grid. We use a custom function to project these coordinates into the ENU system (East-North-Up), based on chapter 3 from grooves (2013, pp. 61 - 72). This turns the location data based on degrees into a 3D coordinate system where every unit represents exactly one meter, providing a stable foundation for the filter.

Furthermore, a crucial part of the calibration lies in the quaternion base and the covariance matrices (R) for GPS position, GPS-based velocity, and the ZUPT. These parameters are important for the accuracy of the entire pipeline. Consequently, a significant amount of time was spent during the calibration phase on fine-tuning these parameters to achieve the best estimation for all runs. The EKF operates in a continuous loop consisting of two main phases: Prediction and Correction. In the following section, the filter uses the

acceleration and orientation data to guess where the runner will be in the next millisecond. For this, the previously calculated 4D quaternions are converted into a 3D rotation matrix using a custom function `quatToRotMat()`, based on grooves chapter 5.3 (2013, pp. 157 - 162) this acts as a bridge between the phone's orientation and heading into the ENU map.

The most critical calculation in this loop is determining the world acceleration. In this step, the estimated bias is subtracted, and gravity is factored in to get the acceleration into the right direction of movement. It takes the previous position, adds the current velocity, and factors in the acceleration to project the next state. This uses basic physics to estimate the new position and velocity based on the previously calculated world acceleration. Because sensors are never perfect, the filter also tracks its own Uncertainty (Covariance), acknowledging that this guess becomes less reliable the longer it goes without a checkpoint.

At the end of the prediction part, the state transition matrix is prepared. This matrix tells the filter how the current position and velocity should change over time based on the detected acceleration. It is also mainly used to keep a continuous record of the predicted trajectory and the mathematical uncertainty associated with every step. This process ensures that the runner's path is not only based on sensor guesses but is constantly corrected by real-world data in the subsequent update phase.

To narrow this uncertainty and determine the most likely position, the algorithm enters the Correction Phase, which builds a list of measurements (z) and a mapping matrix (H) based on the data available at that specific timeframe. If a GPS signal is available, the filter compares its predicted location to the satellite's coordinates. Additionally, if the runner is moving, the GPS speed and heading are used to fix any errors in the predicted velocity. If the sensors detect a foot strike, the filter is informed that the velocity is exactly zero via the Zero Velocity Update (ZUPT). This is a powerful correction that eliminates the drift caused by sensor noise. If any of these "checkpoints" are found, the Kalman Gain is calculated. This gain acts as a weight for the trust placed in the respective sensors, it has the ability to adapt to the most reliable sensor and prioritize that information. The final state is a blend of the prediction and the real-world measurement, resulting in a position estimation and a corrected sensor bias.

2.5. Elevation calculation

To calculate the elevation, we relied exclusively on the barometer data. During development, it became clear that the elevation data provided by the GPS was not optimal; it suffered from significant inaccuracies, particularly when passing under bridges or through tunnels. These anomalies often caused jumps in the altitude readings, which would have compromised the entire calculation. We explored several alternatives, including a deadband calculation to filter out GPS errors when they deviated too far

from the barometric trend. However, since these hybrid approaches proved unreliable, we decided to use the barometric pressure as the primary source, combined with rigorous preprocessing to ensure stability across all test runs.

According to the Manual of the ICAO Standard Atmosphere (1993, p. 13), the altitude is calculated based on the assumption of a constant vertical temperature lapse rate. The mathematical constants utilized in the algorithm result from the integration of the hydrostatic equation and the perfect gas law using the ICAO standard values for sea-level pressure (1013.25 hPa (P_{ref})) and temperature (288.15 K).

The conversion of barometric pressure to altitude is based on the International Barometric Formula, which is derived from the Pressure Scale Height. As illustrated in the underlying physical relationship between the gas constant, absolute temperature, and gravitational acceleration, the scale height determines the vertical pressure gradient. According to the Manual of the ICAO Standard Atmosphere (1993 p.12). The formula used was then converted with the help of gemini into an easy to use calculation, which was integrated into the following implementation:

$$alt_m = 44330 * (1 - (P_baro / P_ref) ^ {1/5.255})$$

This equation ensures that pressure changes are accurately mapped to metric elevation changes. Although the barometric formula fundamentally depends on the temperature, this project utilizes a constant temperature of 15°C (288.15 K). This methodological decision was made because the primary objective is the calculation of relative elevation changes (Gain/Loss) rather than absolute altitude above sea level. Within the typical duration of a running session (30–60 minutes), local temperature fluctuations are minimal and do not significantly impact the precision of relative height. Furthermore, using a fixed temperature maintained more stability and prevented the introduction of unnecessary noise.

To eliminate high-frequency noise and atmospheric fluctuations, we then apply a massive smoothing window of 60 seconds. Following this, the data is downsampled to one frame every 20 seconds. This heavy downsampling is a deliberate choice: because even minor sensor oscillations on flat terrain can accumulate into significant elevation changes, we prioritize a stable trend over high-frequency updates.

The final and most crucial step is the implementation of a Hysteresis Filter with a 20 cm threshold. This logic ensures that the algorithm only registers a change in altitude if the difference between the current reading and the last reference point exceeds 20 cm. If the change is smaller, it is ignored as noise. While this may result in the loss of very minor undulations in the pavement, it provides a much more robust and realistic estimate of the total Elevation Gain and Loss. The processed altitude is then re-interpolated to

match the common timeline, ensuring that the vertical data is perfectly synced with the horizontal trajectory.

2.6 Weighted Distance calculation

In the final stage of our pipeline, we calculate the total distance and velocity using a Hybrid Distance Calculation. This step serves as a final check by combining the smooth movement data from the Extended Kalman Filter (EKF) with the absolute position data from the GPS. While the EKF effectively filters out sudden sensor jumps, it can drift over time. To fix this, we reintegrated the GPS data as a steady reference, following the multi-sensor reliability principles described by Groves (2013, pp. 561–620)). The algorithm first calculates the Euclidean distance (the straight-line distance between two points) for both the EKF and the GPS datasets. These values are then blended using a 80/20 Gps/EKF weighting factor (0.2). This balance allows us to keep the smooth, natural motion of the EKF while staying tethered to the global accuracy of the satellites, which prevents the calculation from drifting off course.

To make the data even more reliable, we added a Noise cut off. If the estimated velocity drops below 0.1 m/s, the distance for that moment is set to zero. This is essential to stop GPS jitter, from artificially increasing the total distance. This combined method provides our best estimate, resulting in a total distance that is both physically realistic and geographically accurate.

2.7 KPI Calculation and saving

In the final stage, the system extracts key performance indicators (KPIs) to provide a clear summary of the activity. The algorithm calculates the total distance, elevation gain, and elevation loss, while also determining the runner's pace in minutes per kilometer (min/km). To ensure accuracy, a velocity threshold is applied so that time spent standing still does not distort the results and no division by zero occurs. By combining the stabilized distance from the hybrid fusion with the filtered velocity data, the pipeline delivers a precise and reliable overview of the run's metrics.

To document and verify these results, all calculated KPIs are exported to a central CSV file. The system is designed to be smart by detecting if a run already exists in the log, updating only the calculated values while protecting any manually entered Ground Truth references to create a structured database for the entire analysis. Finally, a comparison figure is automatically generated to validate the success of the filtering. This visual analysis includes a horizontal plot comparing the jittery, raw GPS path with the

smooth, fused EKF trajectory, as well as a vertical plot that demonstrates the stability of the barometric altitude algorithm against noisy GPS data and the actual terrain's Ground Truth.

3. Methods

3.1 Measurement Setup

Participants were required to wear comfortable sports attire during data collection. To capture the movement, a standard smartphone was utilized, involving various iPhone models. The device was tightly strapped around the runner's waist to align it with the hip, near the center of mass. This specific placement was chosen to minimize motion artifacts caused by limb swing and to allow the sensors to record the runner's trajectory as a single rigid body, to reduce noise from arm or leg movements. Throughout the duration of the run, the phone was not handled, ensuring that the recorded data strictly reflected the movement of the torso.

The data was obtained using the Phyphox application to log raw sensor data at the maximum available sampling rate for all integrated sensors. This was done to ensure a high temporal resolution for the subsequent sensor fusion and for the detection of stationary phases. The sensors utilized for this project included the Accelerometer, which recorded the force at 100 Hz for velocity estimation, and the Gyroscope, which tracked 3D orientation and rotation also at 100 Hz. The Magnetometer provided data at 100 Hz for heading correction and compass estimation, while the Barometer and GNSS (Global Navigation Satellite System) operated at 1 Hz to provide altitude estimation and absolute position data (Latitude and Longitude), respectively.

The experimental protocol was standardized to maintain data consistency. Participants were required to stand still for at least 10 seconds before and after every run. This allowed the algorithm to calculate the initial sensor bias. Runners were instructed to maintain a steady speed and avoid unnecessary stops during the trials. All datasets were exported in CSV format for analysis.

To ensure the reliability of the barometric data, evaluation runs were conducted under uniform and stable weather conditions. Since barometers are sensitive to atmospheric pressure changes, avoiding volatile weather was essential to prevent external pressure shifts from distorting the calculated elevation profiles.

3.2 Ground Truth Establishment

To analyze the accuracy of the distance and elevation estimates, it was essential to establish valid ground truths. Without these references, calibration would not yield reliable results. The decision was made to split the runs into two categories: one focused on horizontal distance and the other on vertical elevation. For the distance ground truth, a standard athletic track field was used because of its calibrated and known lengths. Since one full lap around the track is exactly 400m, runs of different lengths (e.g., 400m, 800m, 1600m) were performed to calibrate and evaluate the algorithm's distance estimation.

The ground truth for the elevation was established using the geographical information system QGIS. This software allowed for the integration of high-resolution terrain data and a detailed analysis of the routes. Initially, the DGM1 datasets (Digital Terrain Model with 1-meter resolution) were sourced from official state websites, specifically the Hessische Verwaltung für Bodenmanagement und Geoinformation for routes in Hesse and the Bayerische Vermessungsverwaltung for trials conducted in Bavaria. A layer mosaic was created using .tif files, and for every recorded run, a shapefile line layer was drawn on the map using an OpenStreetMap overlay for better visibility of the actual paths. This line was then converted into discrete points using the "points along geometry" function, splitting the path into one point every meter. The elevation data was extracted for every individual point using the "sample raster values" (Rasterwerte abtasten) function to obtain the precise Z-data.

Finally, a script processed the exported CSV data to calculate the final ground truth. In this script, a smoothing window of 5 meters was applied, and all data was rounded to the first decimal. This was necessary because minute changes in elevation readings can lead to enormous discrepancies over long distances. It is important to note that this ground truth method has inherent drawbacks, as features like bridges or tunnels can lead to distortions in the DGM1 data. To mitigate this, very simple routes were chosen for the evaluation to ensure the most accurate ground truth possible. Since calibrated high-end barometers are expensive and impractical for varying routes, this DGM-based approach was the most reliable way to obtain a true elevation reading, despite being an inherently flawed estimate.

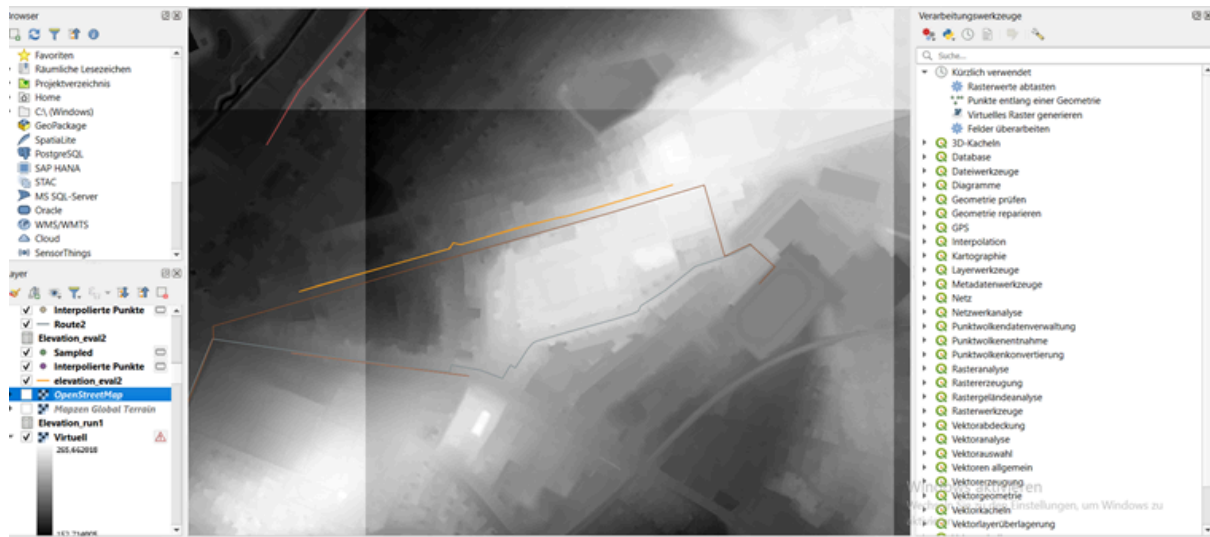


Figure 1: picture of the second evaluation run route for Elevation drawn on the DGM1 mosaic with the functions used to extract the elevation data on the right

3.3 Measurement Phases and Statistical Methodology

This project was divided into two main phases: calibration and evaluation. During the calibration phase, the algorithm pipeline was established using a total of 20 runs. Throughout this phase, the most important parameters, such as the weighting in the loosely coupled fusion, the quaternion base, and the covariance matrices (R) for the gyroscope and accelerometer for the EKF, were constantly adjusted and fine-tuned until the pipeline delivered reasonable results across all datasets. To counteract the risk of overfitting, the evaluation phase used a separate, "frozen" set of 12 runs where all parameters remained unchanged.

To analyze the reliability of the pipeline, three primary statistical methods were applied:

- **Root Mean Square Error (RMSE):** Used to quantify the overall magnitude of the error in physical units (meters). It is particularly sensitive to large outliers, making it an excellent metric for detecting "explosions" in EKF velocity integration.
- **Normalized RMSE (nRMSE) and MARE:** These relative metrics were chosen to allow for a fair comparison between runs of different lengths (e.g., comparing a 400m track run to a 1600m run) by expressing the error as a percentage of the total magnitude.
- **Bland-Altman Analysis:** This method was implemented to assess the "agreement" between the algorithm and the ground truth. Unlike simple error averages, it visualizes systematic bias (e.g., if the algorithm consistently overestimates distance) and the 95% limits of agreement, providing insight into the stability and repeatability of the sensor fusion across different environments.

4. Results

4.1 Quantitative Evaluation Against Ground Truth

The overall accuracy performance of the proposed algorithm was first evaluated across all validation runs. The global accuracy report provides aggregated accuracy percentages together with absolute RMSE values for horizontal distance as well as elevation gain and elevation loss.

--- Global Accuracy Evaluation Report (Updated with RMSE & Loss Error) ---

Run Name	DistAcc%	DistErr (m)	GainAcc%	GainErr (m)	LossAcc%	LossErr (m)
Run1_ELE1	N/A	N/A	89.67	1.36	89.08	1.44
Run1_ELE2	N/A	N/A	96.72	0.85	98.09	0.49
Run2_ELE1	N/A	N/A	72.69	3.61	75.15	3.28
Run2_ELE2	N/A	N/A	98.11	0.49	98.33	0.43
Run3_ELE1	N/A	N/A	88.10	1.57	91.69	1.10
Run3_ELE2	N/A	N/A	94.31	1.47	99.28	0.19
Run1_Dist1200	98.65	16.22	N/A	N/A	N/A	N/A
Run1_Dist1600	98.87	18.15	N/A	N/A	N/A	N/A
Run1_Dist800	97.98	16.16	N/A	N/A	N/A	N/A
Run2_Dist1200	98.65	16.22	N/A	N/A	N/A	N/A
Run2_Dist800	95.20	38.38	N/A	N/A	N/A	N/A
Run2_Dist1600	97.46	40.58	N/A	N/A	N/A	N/A
----- GLOBAL SUMMARY -----						
Distance:	Accuracy: 97.80%	RMSE: 26.571 m (n=6)				
Elev. Gain:	Accuracy: 89.93%	RMSE: 1.845 m (n=6)				
Elev. Loss:	Accuracy: 91.94%	RMSE: 1.555 m (n=6)				

Table 1: Global accuracy evaluation report across all experimental runs (n = 6), including accuracy percentages and RMSE values for distance, elevation gain, and elevation loss.

As shown in Table 1, the algorithm achieves a high horizontal distance accuracy of 97.80%, with an RMSE of 26.571 m. For the vertical component, elevation gain and loss accuracies reach 89.93% and 91.94%, respectively, with RMSE values in the range of approximately 1–2 m. These results confirm robust performance across multiple runs, while indicating slightly reduced accuracy for elevation estimation compared to horizontal distance.

While the global accuracy report summarizes absolute performance in physical units, RMSE values are inherently scale-dependent and therefore do not always allow direct comparison across datasets of different magnitudes. For this reason, additional relative error metrics were computed, including the normalized RMSE (nRMSE), maximum absolute deviation, and

Bayes-corrected RMSE. Furthermore, these metrics enable a direct comparison between the final algorithm and alternative estimation approaches.

Dataset	Metric	nRMSE (%)	Max <u>Error</u> (m)	RMSE BayesCorr (m)
Final Evaluation	Distance	1.48	29.06	NaN
	Elevation Gain	9.46	3.61	1.55
	Elevation Loss	7.97	3.28	1.19
EKF_100%	Distance	4.94	69.31	NaN
	Elevation Gain	105.09	25.80	20.47
	Elevation Loss	615.79	203.81	120.07
Baro + GPS Fusion	Distance	1.88	36.82	NaN
	Elevation Gain	9.50	3.61	1.56
	Elevation Loss	8.02	3.29	1.20

Table 2: Quantitative comparison to ground truth across different estimation methods using normalized RMSE (nRMSE), maximum error, and Bayes-corrected RMSE.

Table 2 demonstrates that the final algorithm configuration provides the best overall agreement with ground truth measurements. The horizontal distance estimation achieves an nRMSE of 1.48%, with a maximum deviation of 29.06 m.

For the vertical component, elevation gain and loss errors remain below 10% nRMSE, with Bayes-corrected RMSE values of approximately 1.55 m and 1.19 m, respectively.

In contrast, the EKF baseline exhibits substantially larger deviations, particularly in elevation loss, where maximum errors exceed 200 m, indicating limited robustness of EKF-only estimation in the evaluated scenarios. The barometric and GPS fusion approach achieves performance closer to the final algorithm, but does not surpass its overall accuracy.

4.2 Bland–Altman Agreement Analysis

In addition to the statistical accuracy metrics reported in Tables 4.1 and 4.2, Bland–Altman plots were generated to further assess the agreement between the estimated values and the ground truth reference measurements. Bland–Altman analysis provides complementary insight by visualizing the mean difference (systematic bias) and the 95% limits of agreement (± 1.96 SD), which describe the expected error range between two measurement methods (Bland & Altman, 1986).

The analysis was performed separately for horizontal distance, elevation gain, and elevation loss in order to evaluate both Horizontal and vertical estimation.

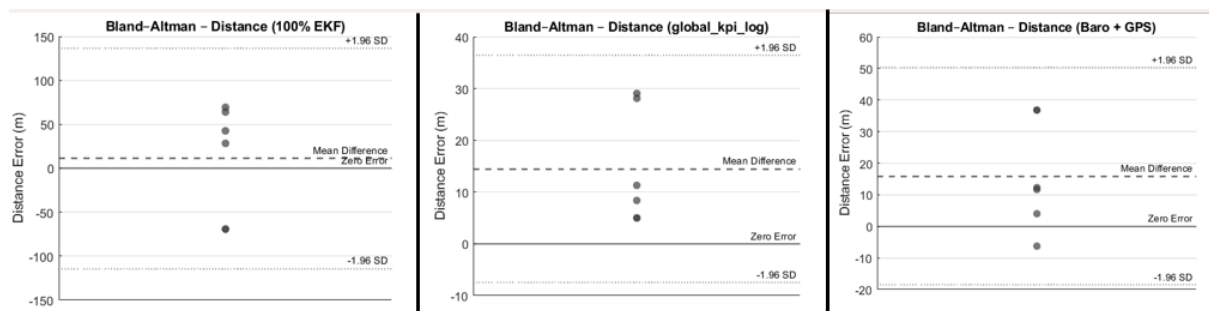


Figure 4.3: Bland–Altman agreement plots for horizontal distance estimation comparing EKF, final algorithm, and Baro+GPS fusion against ground truth.

As shown in Figure 4.3, the EKF baseline exhibits the widest dispersion of horizontal distance errors, with deviations reaching approximately ± 100 m. Several measurements lie far from the zero-error line, indicating poor agreement and high variability.

In contrast, the final algorithm shows a substantially narrower error distribution, with most differences remaining within approximately ± 30 m, confirming improved consistency with the ground truth reference. The Baro+GPS fusion method demonstrates intermediate performance, with deviations extending to roughly ± 40 – 50 m. Overall, the Bland–Altman distance plots support the quantitative RMSE findings reported in Tables 4.1 and 4.2.

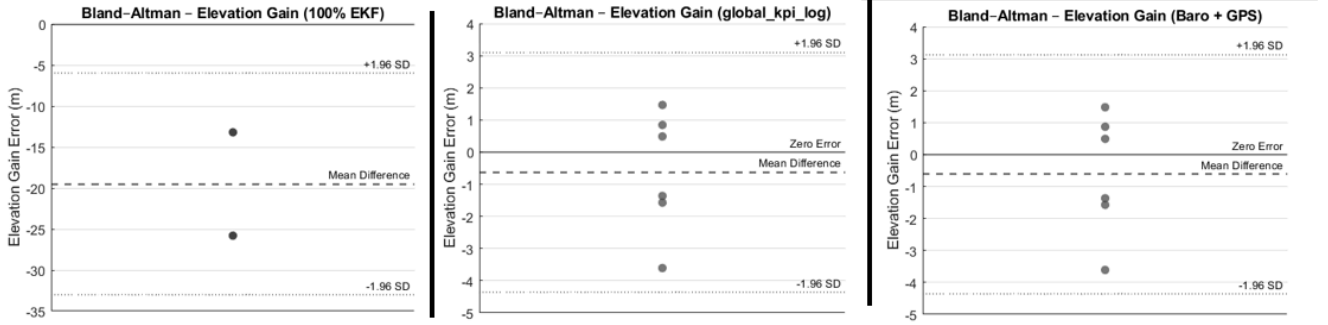


Figure 4.4: Bland–Altman agreement plots for elevation gain estimation across the evaluated methods.

The elevation gain Bland–Altman plots indicate that vertical estimation errors are generally smaller in magnitude than horizontal distance errors but remain sensitive to systematic drift. The EKF baseline shows substantial bias and variability, with elevation gain errors ranging approximately between -30 m and -15 m, indicating consistent underestimation.

In contrast, the final algorithm demonstrates significantly tighter agreement, with most elevation gain differences confined within approximately ± 2 m around the zero-error line, consistent with the low RMSE values reported for vertical gain. The Baro+GPS fusion approach exhibits a similar error range but with slightly increased variability compared to the final method.

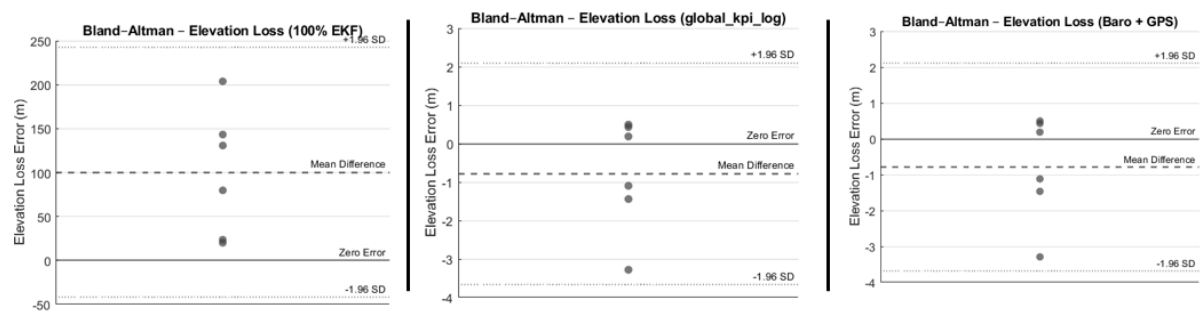


Figure 4.5: Bland–Altman agreement plots for elevation loss estimation across the evaluated methods.

Figure 4.5 reveals the strongest degradation for the EKF baseline in the elevation loss component. Large outliers are visible, with elevation loss errors exceeding **200 m**, indicating severe instability and poor agreement with the reference measurements.

In contrast, the final algorithm maintains a narrow distribution of residual errors, with most differences remaining within approximately ± 3 m, demonstrating reliable vertical loss estimation. The Baro+GPS fusion method again shows intermediate performance, with deviations generally below ± 4 m, but slightly wider dispersion than the final approach.

5. Discussion

The quantitative results presented in Tables 4.1 and 4.2 confirm that the proposed algorithm provides a reliable framework for evaluating horizontal distance accuracy and vertical elevation estimation with respect to ground truth reference measurements. The global accuracy report in Table 4.1 demonstrates that the method achieves strong overall performance in horizontal distance estimation, reaching an accuracy of 97.80% with an RMSE of 26.571 m across all experimental runs. This indicates that the algorithm is capable of reconstructing the horizontal trajectory with only moderate deviations from the reference data, highlighting its robustness in the planar domain.

In comparison, the vertical estimation results reveal slightly reduced accuracy for elevation gain and elevation loss, with RMSE values in the range of approximately 1–2 m. This behavior is expected, since altitude-related measurements are generally more sensitive to systematic drift, environmental influences, and accumulated bias effects than horizontal distance estimation. Such limitations are well documented in multisensor navigation systems, where vertical positioning typically exhibits higher uncertainty due to weaker geometric constraints and increased susceptibility to noise sources (Groves, 2013).

While Table 4.1 provides an absolute summary of algorithm performance in physical units, Table 4.2 extends the evaluation by introducing normalized error measures and a comparison across different estimation approaches. RMSE represents an absolute error metric and is dependent on the magnitude of the measured quantity, meaning that values cannot always be compared directly across datasets with different trajectory scales. For this reason, normalized RMSE (nRMSE) was included as a relative percentage-based indicator, enabling a more meaningful comparison of performance under varying experimental conditions (Chai &

Draxler, 2014). The final algorithm configuration achieves a low horizontal nRMSE of 1.48%, confirming high relative accuracy, while vertical gain and loss remain below 10% nRMSE, indicating stable elevation performance despite the increased challenges of altitude estimation.

A key observation from Table 4.2 is the significant degradation of the EKF baseline, particularly in the vertical component, where maximum deviations exceed 200 m. This suggests that EKF-only estimation is insufficiently robust when systematic biases are not adequately modeled, leading to strong error accumulation over time. This limitation is consistent with the known sensitivity of Kalman-based filtering approaches to unmodeled drift and nonlinear error propagation (Kalman, 1960). In contrast, the fusion-based method provides improved stability, but the final algorithm consistently delivers the best balance between horizontal and vertical accuracy.

Furthermore, the inclusion of Bayes-corrected RMSE values highlights the importance of probabilistic correction strategies in reducing residual systematic errors, especially for elevation-related estimates. Bayesian approaches are widely applied to mitigate systematic bias by incorporating prior knowledge and probabilistic inference into the correction process (Gelman et al., 2013). Such correction strategies align with established measurement uncertainty principles, which emphasize that both random noise and systematic drift must be considered when evaluating the reliability of measurement systems (Joint Committee for Guides in Metrology [JCGM], 2008).

The Bland–Altman plots presented in Figures 4.3–4.5 provide an important extension of the quantitative metrics reported in Tables 4.1 and 4.2, as they reveal not only the magnitude of estimation errors but also their underlying structure in terms of systematic bias and variability across the measurement range (Bland & Altman, 1986). The observed differences between the evaluated methods can be explained by the distinct ways in which each approach handles sensor noise, drift, and error accumulation.

A key finding is the poor agreement of the EKF baseline, particularly visible through the wide limits of agreement and the presence of strong outliers in both horizontal distance and, more critically, elevation loss. This behavior is consistent with the fact that Kalman-based filters rely heavily on accurate process and measurement noise models. When systematic

biases or nonlinear effects are insufficiently modeled, the filter may accumulate errors over time, leading to unstable estimates and large deviations from ground truth (Kalman, 1960). This effect becomes especially pronounced in the vertical component, where small sensor biases can integrate into large altitude errors.

The increased dispersion observed in the elevation gain and elevation loss plots compared to horizontal distance is expected due to the inherently higher uncertainty of vertical positioning. Altitude estimation is typically more sensitive to atmospheric disturbances, sensor drift, and weaker geometric constraints, which makes vertical errors harder to correct than planar errors (Groves, 2013). In particular, elevation loss appears more affected by accumulated drift, which explains the extreme deviations observed for the EKF baseline.

In contrast, the final algorithm demonstrates substantially narrower agreement limits and reduced systematic deviation. This improvement can be attributed to the applied correction strategy, which mitigates residual bias effects and reduces long-term drift contributions. The inclusion of Bayesian correction (BayesCorr) is especially relevant in this context, as probabilistic bias estimation allows systematic error components to be compensated through inference rather than treated as purely random noise (Gelman et al., 2013). As a result, the final approach achieves more stable agreement with ground truth across both horizontal and vertical domains.

From a measurement science perspective, these findings highlight the importance of accounting for both random noise and systematic uncertainty sources when evaluating positioning accuracy. Established uncertainty frameworks emphasize that drift and bias effects must be explicitly addressed in order to obtain reliable measurement performance (Joint Committee for Guides in Metrology [JCGM], 2008). Overall, the Bland–Altman analysis confirms that the proposed algorithm not only reduces average error metrics but also improves consistency and robustness by limiting systematic deviations, particularly in the challenging vertical estimation components.

6. Limitation

The performance of the proposed algorithm is influenced by several factors, including the inherent noise and drift of smartphone sensors, environmental conditions affecting GNSS and barometric measurements, and the need for careful calibration of fusion parameters. In addition, the relatively limited number of evaluation runs and the use of a waist-mounted device placement may restrict the generalizability of the findings.

7. Conclusion

This study demonstrated that integrating GNSS with inertial and barometric sensors significantly improves the estimation of horizontal distance and elevation compared to single-sensor approaches. The proposed sensor fusion pipeline achieved reliable performance across multiple running scenarios, particularly in mitigating drift and systematic bias. Overall, the results highlight the effectiveness of multi-sensor fusion for enhancing accuracy in smartphone-based movement tracking.

6. Literature

Chiang, K. W., Duong, T. T., & Liao, J. K. (2013). The performance analysis of a MEMS-based INS/GPS integrated system with different smoothing algorithms. *Sensors (Basel, Switzerland)*, 13(6), 7736–7755. <https://doi.org/10.3390/s130809549>

Groves, P. D. (2013). *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems* (2nd ed.). Artech House.

Grewal, M. S., Weill, L. R., & Andrews, A. P. (2007). *Global positioning systems, inertial navigation, and integration* (2nd ed.). Wiley

Hessische Verwaltung für Bodenmanagement und Geoinformation. (n.d.). *Digitale Geländemodelle*. Accessed on the 10. December 2025 <https://hvb.g.hessen.de/landesvermessung/geotopographie/3d-daten/digitale-gelaendemodelle>

Hu, X., Zhang, Y., & Li, L. (2023). Evaluation of smartphone GNSS positioning accuracy in diverse urban environments. *Journal of Navigation and Positioning Research*, 11(2), 45–58. <https://doi.org/10.1186/s43020-025-00183-8>

Bayerische Vermessungsverwaltung. (n.d.). *OpenData: Digitales Geländemodell 1m (DGMI)*. Accessed on the 3. January 2026 <https://geodaten.bayern.de/opengeodata/OpenDataDetail.html?pn=dgm1>

Madgwick, S. O. (2010). *An efficient orientation filter for inertial and inertial/magnetic sensor arrays*. University of Bristol, Report. <https://x-io.co.uk/open-source-imu-and-ahrs-algorithms/>

International Civil Aviation Organization. (1993). *Manual of the ICAO Standard Atmosphere: extended to 80 kilometres (262 500 feet)* (Document 7488/2, 3rd ed.).

Skog, I., Handel, P., Nilsson, J. O., & Rantakokko, J. (2010). Zero-velocity detection—An algorithm evaluation. *IEEE Transactions on Biomedical Engineering*, 57(11), 2657–2666. <https://doi.org/10.1109/TBME.2010.2060723>